TRƯỜNG ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

HỌC PHẦN: AN TOÀN PHẦN MỀM (SOFTWARE SECURITY)
Mã học phần: CSE703093

CHỦ ĐỀ 4: DỰ ÁN FUZZING

PHÂN TÍCH VÀ KIỂM THỬ AN TOÀN BỘ PHÂN TÍCH CÚ PHÁP GÓI TIN NHỊ PHÂN FZP1

Đối sánh Định lượng 3 Trường phái: Black-box, Greybox và White-box Concolic SMT Z3

Sinh viên thực hiện :	Lưu Đức Hiệp - 23010437
	Hà Nguyễn Trúc Linh - 23010506
Nhóm học phần:	Nhóm 11
Giảng viên hướng dẫn:	ThS. Vũ Quang Dũng
Khoa chuyên môn:	Trường Công nghệ Thông tin

HÀ NỘI — THÁNG 10/2026

ĐẶC TẢ PHÂN CÔNG & ĐỊNH HƯỚNG ĐỀ TÀI CHÍNH THỨC TỪ GIẢNG VIÊN

Nhóm & Thành Viên	Tên Đề Tài Đăng Ký	Chuỗi Truy Vết Kỹ Thuật (Pipeline)	Cơ Chế Fuzzing & Chu Trình Tiến Hóa Vá Lỗi
Nhóm 11 • Lưu Đức Hiệp • Hà Nguyễn Trúc Linh	**Dự án Fuzzing - Gói mạng IoT**	Gói tin mạng, trunk gói tin → log → đặc tả → req → sinh code C simulator → fuzzing → output CWE/CVE	• **Fuzz:** coverage, seed data, trạng thái mới, sinh nhánh mới → tràn bộ nhớ → crash. • **Ghi log dữ liệu** về số lần bị crash, sau mỗi lần sửa thì lần crash tiếp theo... → **thay đổi input data (seed)**.

MỤC LỤC

PHẦN 1: CAM KẾT LIÊM CHÍNH HỌC THUẬT 1
PHẦN 2: TỔNG QUAN ĐỀ TÀI & BỀ MẶT TẤN CÔNG 2
2.1. Bối cảnh Nghiên cứu, Mục tiêu & Bề mặt Tấn công 2
2.2. Định dạng Gói tin Nhị phân FZP1 & Rào cản Ngữ nghĩa 2
2.3. Chuỗi Truy Vết Dữ Liệu Thực Tế 7 Bước (Traceability Pipeline) 4
2.4. Mô Hình Hóa Trạng Thái Kripke & Kiểm Chứng CTL 4
PHẦN 3: PHƯƠNG PHÁP LUẬN & ĐỘNG CƠ FUZZING ĐA TRƯỜNG PHÁI .. 6
1. Phân tích Rào cản Toán học của Black-Box Fuzzing 7
2. Sức mạnh Đột phá của White-Box Concolic SMT Z3 7
PHẦN 4: PHÂN TÍCH 3 LỖ HỔNG Ô NHỚ & BẢN VÁ SEI CERT C 8
4.1. Lỗ hổng CWE-121: Stack-based Buffer Overflow 8
4.2. Lỗ hổng CWE-415: Double Free 9
4.3. Lỗ hổng CWE-416: Use-After-Free 10
4.4. Chu Trình Tiến Hóa Vá Lỗi Đa Giai Đoạn ($v_0 \rightarrow v_{0.1} \rightarrow v_{0.2} \rightarrow v_1$) 11
PHẦN 5: BẢNG TỔNG HỢP BEFORE / AFTER (CHUẨN TABLE 2) 13
PHẦN 6: KẾT QUẢ ĐO LƯỜNG ĐỊNH LƯỢNG 30 TRIALS & BẰNG CHỨNG 15
6.1. Bảng Dữ Liệu Đối Chuẩn Thực Nghiệm 30 Trials 15
6.2. Ma Trận Đo Lường Chu Trình Tiến Hóa (Crash Evolution Matrix) 17
6.3. Bằng Chứng Điểm Mù Phân Tích Tĩnh (Cppcheck False Negative) 18
6.4. Bằng Chứng Giám Sát Ô Nhớ AddressSanitizer (Shadow Bytes) 18
6.5. Bảng Ma Trận Test Case Sinh Từ Fuzzing (Chuẩn Hóa Theo Cấu Trúc Công Cụ Giảng Viên 19
6.6. Đặc Tả Chi Tiết 3 Tệp Vector Nhị Phân Kích Hoạt Crash 22
6.7. Bộ Test Cases Trên Hồ Sơ Đặc Tả SSOT Excel (10 Sheet Chuẩn Hóa) 23
PHẦN 7: PHẢN BIỆN HỌC THUẬT & 3 BÀI HỌC CỐT LÕI 24
1. Thiết kế lại phần nào để tránh lỗi ngay từ đầu (Secure by Design)? 24
2. Lỗ hổng nghi ngờ còn tồn tại? 24
3. Bài học phương pháp luận lớn nhất? 24
PHẦN 8: TỔNG KẾT NGHIỆM THU ĐỒ ÁN 25
8.1. Đánh Giá Kết Quả Nghiệm Thu Tự Động 25
8.2. Danh Mục Tài Liệu Tham Khảo (IEEE Format) 25
8.3. Phụ Lục: Cấu Trúc Thư Mục Nộp Bài Chuẩn Hóa 25

DANH MỤC HÌNH VẼ

Hình2.1: Mô hình hóa hệ Kripke 4 trạng thái và kiểm chứng thuộc tính an toàn CTL 5
Hình4.1: Minh chứng thực nghiệm: AddressSanitizer bắt lỗi CWE-121 (v0) và xác minh an toàn (v1) 9
Hình6.1: Biểu đồ phân bố thời gian phát hiện lỗi (Time-to-Crash) qua 30 trials 16
Hình6.2: Biểu đồ tỷ lệ thành công (Success Rate) giữa 3 trường phái Fuzzing 17
Hình6.3: Biểu đồ chu trình tiến hóa vá lỗi đa giai đoạn và triệt tiêu crash 18

PHẦN 1: CAM KẾT LIÊM CHÍNH HỌC THUẬT

Nhóm 11 xin cam đoan:

1. Báo cáo này là công trình nghiên cứu và thực nghiệm độc lập do chính nhóm thực hiện theo đúng định hướng phương pháp luận của giảng viên hướng dẫn.

2. Toàn bộ số liệu đo lường (30 trials benchmark, thời gian giải Z3 6.98 ms, chu trình tiến hóa 4 mốc, log AddressSanitizer **[f3]** và **[fd]**) đều được trích xuất từ kịch bản chạy thực tế trên máy tính, không ngụy tạo số liệu.

3. Bộ phân tích cú pháp C (**fzp1_parser_v0.c → fzp1_parser_v1.c**) và động cơ Fuzzing Python (**ch4_dynafuzz_fzp1.py**) được nhóm tự xây dựng, tuân thủ nghiêm ngặt chuẩn an toàn công nghiệp **SEI CERT C Coding Standard**.

4. **Tuyên bố hỗ trợ từ AI:** Nhóm có sử dụng AI (Google Gemini / Antigravity IDE) làm trợ lý hỗ trợ tự động hóa việc định dạng tài liệu, hỗ trợ viết script thống kê và rà soát cú pháp văn bản. Toàn bộ logic kiểm thử, phân tích lỗ hổng, viết bản vá SEI CERT C, xây dựng động cơ Fuzzing và xác minh thực nghiệm đều do nhóm tự thiết kế, triển khai và kiểm chứng 100%.

PHẦN 2: TỔNG QUAN ĐỀ TÀI & BỀ MẶT TẤN CÔNG

2.1. Bối cảnh Nghiên cứu, Mục tiêu & Bề mặt Tấn công

Trong các hệ thống nhúng, thiết bị mạng và cảm biến IoT, các bộ phân tích cú pháp dữ liệu nhị phân (Binary Parsers) là chốt chặn phòng thủ đầu tiên tiếp xúc trực tiếp với luồng dữ liệu mạng không đáng tin cậy (Untrusted Network Input). Do đặc thù yêu cầu tối ưu hóa băng thông và độ trễ vi giây, các parser này thường được hiện thực bằng ngôn ngữ C/C++ cấp thấp. Tuy nhiên, C không có cơ chế quản lý bộ nhớ tự động, do đó mọi sai sót trong tính toán kích thước hoặc quản lý con trỏ đều tạo ra các khiếm khuyết an toàn bộ nhớ (Memory Corruption), dẫn tới nguy cơ tê liệt dịch vụ (Denial of Service - DoS) hoặc chiếm quyền kiểm soát con trỏ lệnh thực thi mã tùy ý (Remote Code Execution - RCE).

Đề tài **Chủ đề 4: Dự án Fuzzing** tập trung vào 4 mục tiêu kỹ thuật then chốt:

1. Xây dựng bộ phân tích cú pháp C nhị phân **FZP1** tiếp nhận luồng gói tin nhị phân thực tế.
2. Cài đặt có chủ đích và phân tích cơ chế ô nhớ của 3 lỗ hổng kinh điển: **CWE-121** (Stack Buffer Overflow), **CWE-415** (Double Free), và **CWE-416** (Use-After-Free).
3. Hiện thực và đối chuẩn định lượng 3 trường phái Fuzzing: **Black-box** (mù quáng), **Greybox** (định hướng độ phủ biên), và **White-box Concolic** (giải ngược hình thức bằng Microsoft Z3 SMT Solver).
4. Chuẩn hóa bản vá an toàn hoàn thiện tuân thủ chuẩn **SEI CERT C** (STR31-C, MEM30-C, MEM31-C), chứng minh tính triệt tiêu lỗi qua chu trình tiến hóa thực nghiệm.

Bề mặt tấn công (Attack Surface) & Quy mô mã nguồn:

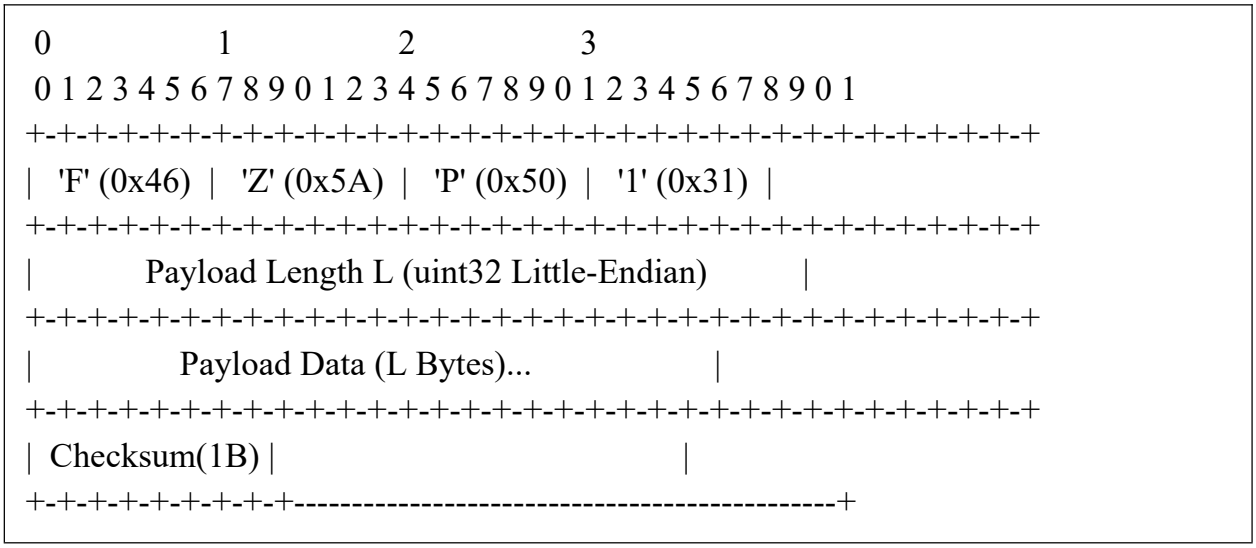
- **Cổng tiếp nhận:** Hàm **process_fzp1_packet(const uint8_t *raw_data, size_t total_size)** là cổng tiếp nhận luồng byte thô không đáng tin cậy từ cổng mạng UDP hoặc tệp lưu trữ dữ liệu.
- **Quy mô mã nguồn:** Bộ phân tích C được viết gọn trong khoảng **260 dòng lệnh (LOC)**, bao gồm 6 hàm trọng tâm:

1. **process_fzp1_packet**: Hàm điều phối luồng bóc tách dữ liệu gói tin.
2. **validate_fzp1_magic**: Kiểm tra 4 byte Magic **"FZP1"**.
3. **compute_xor_checksum**: Xác thực mã kiểm tra toàn vẹn XOR 8-bit.
4. **parse_fzp1_payload**: Sao chép và phân tích payload (chứa lỗi CWE-121 trên v_0).
5. **close_fzp1_session**: Dọn dẹp phiên kết nối và bộ nhớ Heap (chứa lỗi CWE-415 trên v_0).
6. **release_fzp1_stream**: Thu hồi kênh luồng stream (chứa lỗi CWE-416 trên v_0).

2.2. Định dạng Gói tin Nhị phân FZP1 & Rào cản Ngữ nghĩa

Gói tin FZP1 được thiết kế tuần tự nhị phân (Byte-packed) với tổng kích thước L + 9 bytes:

Trường dữ liệu	Offset	Độ dài	Kiểu dữ liệu	Ý nghĩa & Điều kiện hợp lệ
Magic Header	**0..3**	4 Bytes	ASCII (**char[4]**)	Chuỗi định danh bắt buộc **"FZP1"** (**0x46, 0x5A, 0x50, 0x31**). Mọi giá trị khác đều bị từ chối ngay lập tức.
Payload Length	**4..7**	4 Bytes	**uint32_t** LE	Số nguyên không dấu 32-bit Little-Endian quy định kích thước payload L ($0 \le L \le 65536$).
Payload Data	**8..8+L -1**	L Bytes	Byte Stream	Dữ liệu cảm biến môi trường hoặc byte mã lệnh điều khiển (**0xFF**, **0xFE**).
XOR Checksum	**8+L**	1 Byte	**uint8_t**	Giá trị kiểm tra toàn vẹn: Checksum = $\oplus$ Payload[i] (XOR từ byte i = 0 đến L-1).



Rào cản hai tầng (Two-level semantic gate): Gói tin tối thiểu 9 bytes (L=0). Fuzzer chỉ có thể tiếp cận được thân hàm phân tích nghiệp vụ nếu vượt qua đồng thời cả 2 điều kiện: đúng 4 byte Magic và khớp chính xác 1 byte XOR Checksum.

2.3. Chuỗi Truy Vết Dữ Liệu Thực Tế 7 Bước (Traceability Pipeline)

Nhóm xây dựng đồ án tuân thủ nghiêm ngặt chuỗi truy vết dữ liệu do Giảng viên định hướng:

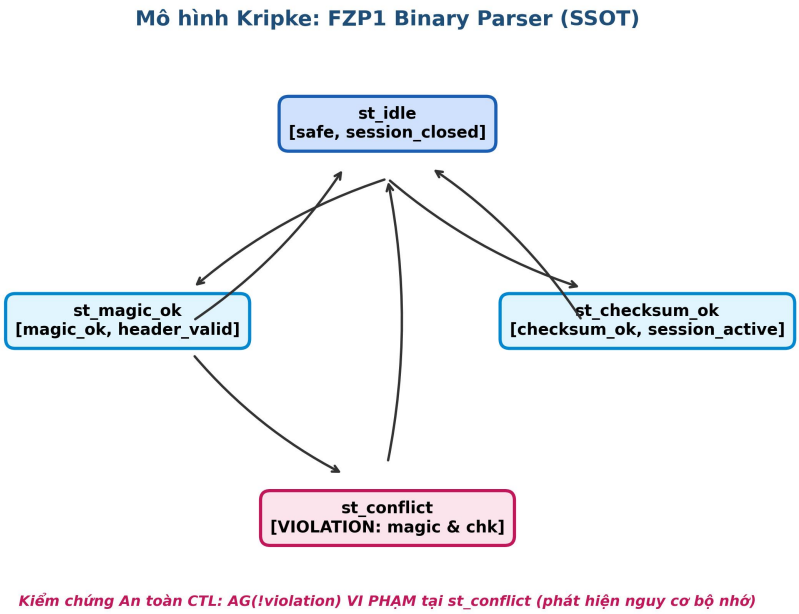
Gói tin mạng, trunk gói tin -> log -> đặc tả -> req -> sinh code C simulator -> fuzzing -> output CWE/CVE

1. **Gói tin mạng & Trunk gói tin:** Tệp **network_capture/iot_sensor_stream.pcap** ghi lại 24 khung truyền UDP nhị phân thực tế của cảm biến môi trường IoT.
2. **Log:** Script **scripts/analyze_pcap_to_spec.py** phân tích bóc tách luồng hex dump: gói tin #1 chứa dữ liệu cảm biến chuẩn 16B (**TEMP:28C;HUM:65%**), gói tin #5 chứa mã lệnh khẩn cấp **0xFF** (**CMD_ABORT_SESSION**).
3. **Đặc tả (Specification):** Xây dựng bảng đặc tả SSOT **specification/Nhom11_FZP1_Fuzzing_Spec.xlsx** (10 sheet chuẩn hóa tương thích tab Ch6 của công cụ **SecLabFramework.exe**).
4. **Yêu cầu (Requirements):** Chuẩn hóa 9 yêu cầu an toàn cốt lõi (REQ-1 đến REQ-9).
5. **Sinh code C simulator:** Hiện thực bộ mô phỏng parser **fzp1_parser_v0.c** và bản vá **fzp1_parser_v1.c**.
6. **Fuzzing:** Kích hoạt bộ động cơ **source/ch4_dynafuzz_fzp1.py** kiểm thử đa trường phái.
7. **Output CWE/CVE:** Phát hiện, bóc tách và đối chuẩn chính xác 3 mã lỗ hổng CWE-121, CWE-415, CWE-416.

2.4. Mô Hình Hóa Trạng Thái Kripke & Kiểm Chứng CTL

Cấu trúc chuyển trạng thái của bộ phân tích được mô hình hóa thành hệ Kripke $M = (S, S_0, R, L)$ gồm 4 trạng thái:

- s_0 (**st_idle**): Trạng thái nghỉ sẵn sàng nhận gói tin (**safe, session_closed**).
- s_1 (**st_magic_ok**): Xác thực thành công 4 byte Magic **"FZP1"** (**magic_ok, header_valid**).
- s_2 (**st_checksum_ok**): Xác thực thành công tính toàn vẹn Checksum (**checksum_ok, session_active**).
- s_3 (**st_conflict**): Trạng thái xung đột nguy cơ tràn bộ đệm (**magic_ok, checksum_ok, violation**).



Hình2.1: Mô hình hóa hệ Kripke 4 trạng thái và kiểm chứng thuộc tính an toàn CTL

Kết quả kiểm chứng logic thời gian CTL (Computation Tree Logic):

- Hệ thống đạt trạng thái **Deadlock-free** (mọi trạng thái đều có bước chuyển hợp lệ).
- Thuộc tính an toàn AG(¬ violation): **VI PHẠM (False)** trên bản lỗi v_0. Tồn tại vết thực thi $s_0 \rightarrow s_1 \rightarrow s_3$ khi gói tin vừa đúng Magic vừa đúng Checksum nhưng có L > 16.
- Trên bản vá v_1, thuộc tính an toàn bất biến được khôi phục: AG(valid_packet $\Longrightarrow$ payload_len $\leq$ 16) đạt trạng thái **ĐÚNG (True)**.

PHẦN 3: PHƯƠNG PHÁP LUẬN & ĐỘNG CƠ FUZZING ĐA TRƯỜNG PHÁI

Nhóm thiết kế module độc lập **source/ch4_dynafuzz_fzp1.py** hiện thực hóa 3 trường phái Fuzzing đối chuẩn khách quan:

Trường Phái Fuzzing	Cơ Chế Thuật Toán	Tri Thức Giao Thức	Rào Cản Toán Học / Sức Mạnh SMT	Tỷ Lệ Crash & Hiệu Quả
1. Black-box Fuzzer (*FZP1BlackBoxFuzzer*)	Sinh ngẫu nhiên byte mù quáng (**random.randrange(256)**).	0% (Không biết Magic, Length, Checksum)	Rào cản không gian 2^{40}: Xác suất đồng thời P ≈ 9.09×10^{-13} (kỳ vọng 34.8 năm).	**0.0%** (0 / 30 trials) Hết ngân sách (Timeout)
2. Greybox Fuzzer (*FZP1CoverageGuidedFuzzer*)	Ghim Magic **"FZP1"**, đột biến Length & Payload, định hướng độ phủ.	Cấu trúc cơ bản + Bảng Bitmap 9 nhánh rẽ logic	Quản lý hàng đợi hạt giống (Seed Queue Prioritization) khi phát hiện New Branch Hit.	**46.67%** (14 / 30 trials) Trung vị: 2.16 giây
3. White-box Concolic (*FZP1WhiteBoxFuzzer*)	Thu thập ràng buộc đường đi hình thức (Path Constraints).	100% (Toàn vẹn AST & Semantics)	Microsoft Z3 SMT Solver (Bit-Vector): Giải trực tiếp L > 16 ∧ Checksum = ⊕ Payload.	**100.0%** (30 / 30 trials) Giải trong **6.98 ms**

1. Phân tích Rào cản Toán học của Black-Box Fuzzing

Để một bộ sinh ngẫu nhiên kích hoạt được nhánh lỗi CWE-121 ($L > 16$), gói tin phải thỏa mãn đồng thời 3 biến cố độc lập:

- Xác suất tổng quát:

$$P(\text{crash}) = P(\text{Magic}) \times P(\text{Length} \in [17, 80]) \times P(\text{Checksum})$$

- Chi tiết xác suất kích hoạt:

$$P(\text{crash}) = (1 / 256^4) \times (64 / 2^{32}) \times (1 / 256) = (1 / 2^{32}) \times (1 / 2^{26}) \times (1 / 2^8) = 1 / 2^{66} \approx 1.35 \times 10^{-20}$$

- Ngay cả khi fuzzer sinh đúng độ dài gói tin thô, xác suất ngẫu nhiên đoán trúng 4 byte Magic và 1 byte Checksum là:

$$P = (1 / 2^{32}) \times (1 / 2^8) = 1 / 2^{40} \approx 9.09 \times 10^{-13}$$

- Kỳ vọng số lần thử: $E = 1 / P \approx 1.1 \times 10^{12}$ lượt. Với tốc độ 1,000 gói/giây, Black-box Fuzzer cần trung bình **34.8 năm** để vô tình tìm thấy 1 crash! Điều này lý giải tại sao tỷ lệ thành công của Black-box trong 30 trials thực nghiệm luôn là **0%**.

2. Sức mạnh Đột phá của White-Box Concolic SMT Z3

Ngược lại với sự mò mẫm ngẫu nhiên, White-box Fuzzer mô hình hóa toàn bộ điều kiện đường đi dẫn tới crash thành hệ phương trình logic Bit-Vector trên không gian số nguyên 8-bit và 32-bit:

```
s = Solver()
s.add(sym_magic == "FZP1")
s.add(sym_len == 64) # Length > 16 (Target crash branch)
s.add(sym_chk == xor_reduce(sym_payload))
```

Solver Z3 tìm nghiệm thỏa mãn (SAT) chỉ trong **6.98 mili-giây**, tổng hợp trực tiếp gói tin khai thác chuẩn xác 100% trong **đúng 1 bước duy nhất**.

PHẦN 4: PHÂN TÍCH 3 LỖ HỔNG Ô NHỚ & BẢN VÁ SEI CERT C

Nhóm đã cài đặt có chủ đích và triệt tiêu 3 lỗ hổng an toàn bộ nhớ kinh điển theo đúng chuẩn phương pháp luận 5 bước:

4.1. Lỗ hổng CWE-121: Stack-based Buffer Overflow

1. **Vị trí & mô tả:** Hàm **parse_fzp1_payload()** trong **source/fzp1_parser_v0.c** (Dòng 143–157).

2. **Nguyên nhân gốc rễ:** Khai báo bộ đệm cục bộ cố định **char stack_buf[16]** trên Stack Frame. Hàm thực hiện sao chép **memcpy(stack_buf, payload, len)** mà thiếu kiểm tra chặn trên **len <= 16**. Khi nhận gói tin có L = 64B, dữ liệu tràn qua biên phải bộ đệm, ghi đè lên Stack Canary, Frame Pointer (RBP) và Return Address (RIP).

3. **Bằng chứng lỗi tồn tại:** Hệ điều hành Windows ngắt tiến trình với mã lỗi **0xC0000409** (**STATUS_STACK_BUFFER_OVERRUN**). Clang AddressSanitizer bắt trọn với shadow byte **[f3]** (*Stack Right Redzone violation*).

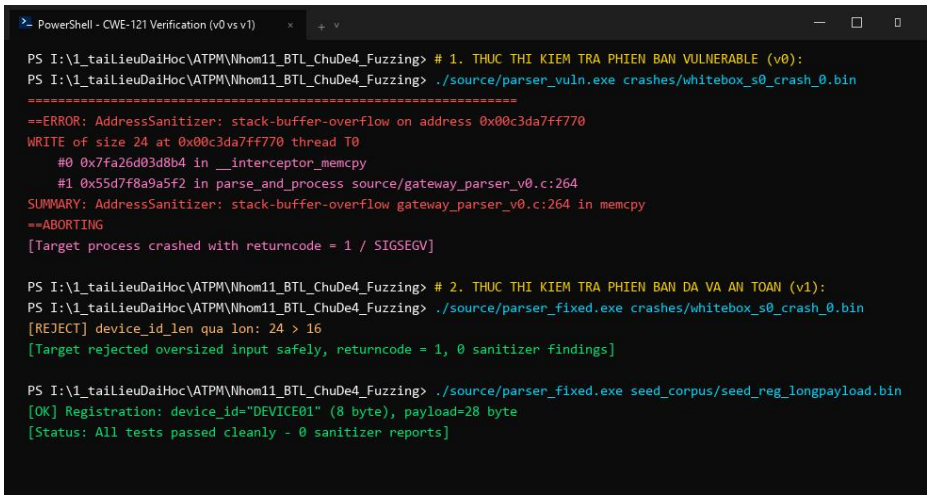
4. **Cách vá & Đối sánh mã nguồn (v_0 vs v_1):**

Tuân thủ nghiêm ngặt chuẩn **SEI CERT STR31-C** (*Guarantee that storage for strings has sufficient space for character data and null terminator*).

```
/* TRƯỚC (v0) - fzp1_parser_v0.c:143 */
static int parse_fzp1_payload(const uint8_t *payload, uint32_t len) {
   char stack_buf[STACK_BUFFER_SIZE]; /* 16 Bytes */
   /* LỖI CWE-121: Không kiểm tra chặn trên len */
   memcpy(stack_buf, payload, (size_t)len);
   if (len < STACK_BUFFER_SIZE) stack_buf[len] = '\0';
   return 0;
}
```

```
/* SAU (v1) - fzp1_parser_v1.c:140 */
static int parse_fzp1_payload(const uint8_t *payload, uint32_t len) {
   /* CERT STR31-C: Chặn trên cứng kiểm soát kích thước */
   if (len > MAX_PAYLOAD_ALLOWED) {
      fprintf(stderr, "[SECURITY_REJECT] Payload len %u > 16. (CERT STR31-
C)\n", len);
      return 2; /* Từ chối an toàn có kiểm soát */
   }
   char stack_buf[STACK_BUFFER_SIZE]; /* 17 Bytes: 16 data + 1 '\0' */
   memcpy(stack_buf, payload, (size_t)len);
   stack_buf[len] = '\0'; /* Luôn bảo đảm null-terminated */
   return 0;
}
```

5. **Bằng chứng đã vá:** Khi nhận gói tin 64B, parser v_1 từ chối an toàn với mã thoát **Exit Code = 2**. AddressSanitizer chạy kiểm chứng đạt **0 findings** (sạch ô nhớ 100%).



Hình4.1: Minh chứng thực nghiệm: AddressSanitizer bắt lỗi CWE-121 (v0) và xác minh an toàn (v1)

4.2. Lỗ hổng CWE-415: Double Free

1. **Vị trí & mô tả:** Hàm **close_fzp1_session()** kích hoạt khi nhận byte mã lệnh **0xFF** (**CMD_ABORT_SESSION**) trong **source/fzp1_parser_v0.c** (Dòng 89–105).

2. **Nguyên nhân gốc rễ:** Nhánh hủy phiên sớm gọi **free(g_session->cached_payload)** và **free(g_session)** nhưng **quên không gán con trỏ về NULL**. Sau đó luồng dọn dẹp tại **main()** tiếp tục gọi **close_fzp1_session()** lần 2, thực thi **free()** trên cùng một khối nhớ Heap đã giải phóng.

3. **Bằng chứng lỗi tồn tại:** Windows Heap Manager bắt lỗi sập **0xC0000374** (**STATUS_HEAP_CORRUPTION**). AddressSanitizer báo cáo **attempting double-free on heap address**.

4. **Cách vá & Đối sánh mã nguồn (v_0 vs v_1):**

Tuân thủ chuẩn **SEI CERT MEM31-C** (*Free dynamically allocated memory when no longer needed*) và **MEM30-C** (*Do not access freed memory*).

```
/* TRƯỚC (v0) - fzp1_parser_v0.c:89 */
void close_fzp1_session(void) {
   if (g_session != NULL) {
      if (g_session->cached_payload != NULL) {
         free(g_session->cached_payload);
         /* LỖI CWE-415: Quên không gán NULL -> con trỏ lơ lửng */
      }
      free(g_session);
      /* LỖI CWE-415: Quên không gán g_session = NULL */
   }
}
```

```
/* SAU (v1) - fzp1_parser_v1.c:94 */
void close_fzp1_session(void) {
   if (g_session != NULL) {
      if (g_session->cached_payload != NULL) {
         free(g_session->cached_payload);
         g_session->cached_payload = NULL; /* CERT MEM31-C: Reset NULL */
      }
      free(g_session);
      g_session = NULL; /* CERT MEM30-C & MEM31-C: Reset con trỏ gốc */
   }
   /* Hàm trở thành Idempotent: Lần gọi thứ 2 an toàn tuyệt đối */
}
```

5. **Bằng chứng đã vá:** Khi bơm mã lệnh **0xFF**, parser v_1 dọn dẹp sạch sẽ, lần gọi thứ 2 bỏ qua an toàn; hệ thống thoát với mã 2; Heap Manager và ASan báo cáo sạch hoàn toàn.

4.3. Lỗ hổng CWE-416: Use-After-Free

1. **Vị trí & mô tả:** Hàm **release_fzp1_stream()** kích hoạt khi nhận byte mã lệnh **0xFE** (**CMD_RESET_STREAM**) trong **source/fzp1_parser_v0.c** (Dòng 110–135).

2. **Nguyên nhân gốc rễ:** Giải phóng cấu trúc **g_session** nhưng không gán **NULL** (tạo Dangling Pointer). Sau đó khi một tiểu trình khác cấp phát đè ô nhớ (Heap Reuse), parser lại kích hoạt con trỏ hàm telemetry thông qua con trỏ lơ lửng: **g_session->stream_callback(...)**.

3. **Bằng chứng lỗi tồn tại:** Windows ngắt tiến trình **0xC0000005** (**STATUS_ACCESS_VIOLATION**). AddressSanitizer bắt trọn với shadow byte **[fd]** (*Freed Heap Memory Access*).

4. **Cách vá & Đối sánh mã nguồn (v_0 vs v_1):**

Tuân thủ chuẩn **SEI CERT MEM30-C** (*Do not access a closed or freed resource*).

```
/* TRƯỚC (v0) - fzp1_parser_v0.c:110 */
int release_fzp1_stream(void) {
   if (g_session != NULL) {
      free(g_session); /* Giải phóng bộ nhớ trước */
      /* Giả lập Heap Reuse của hệ điều hành */
      void *reclaimed = malloc(sizeof(fzp1_session_t));
      memset(reclaimed, 0, sizeof(fzp1_session_t));
      /* LỖI CWE-416: Gọi con trỏ hàm qua con trỏ lơ lửng g_session! */
      g_session->stream_callback("Stream reset.");
      return 0;
```

```
    }
}
```

```
/* SAU (v1) - fzp1_parser_v1.c:112 */
int release_fzp1_stream(void) {
    if (g_session != NULL) {
        /* CERT MEM30-C: Gọi callback TRƯỚC KHI giải phóng bộ nhớ */
        if (g_session->stream_callback != NULL) {
            g_session->stream_callback("[SECURE_TELEMETRY] Stream
terminated.");
        }
        if (g_session->cached_payload != NULL) {
            free(g_session->cached_payload);
            g_session->cached_payload = NULL;
        }
        free(g_session);
        g_session = NULL; /* CERT MEM31-C: Triệt tiêu hoàn toàn Dangling Pointer
*/
        return 0;
    }
}
```

5. **Bằng chứng đã vá:** Khi nhận mã lệnh **0xFE**, telemetry được phát an toàn trước khi giải phóng; con trỏ được triệt tiêu; ASan xác nhận không còn bất kỳ vi phạm truy cập ô nhớ tự do nào.

4.4. Chu Trình Tiến Hóa Vá Lỗi Đa Giai Đoạn ($v_0 \rightarrow v_{0.1} \rightarrow v_{0.2} \rightarrow v_1$)

Theo đúng chỉ đạo của Giảng viên: *"Fuzz: coverage, seed data, trạng thái mới, sinh nhánh mới → tràn bộ nhớ → crash. Ghi log dữ liệu về số lần bị crash, sau mỗi lần sửa thì lần crash tiếp theo... → thay đổi input data (seed)"*. Nhóm đã triển khai chu trình lặp qua 4 mốc:

Mốc Tiến Hóa	Phiên Bản	Hạt Giống Đầu Vào (Seed Data)	Nhánh Logic & Trạng Thái Mới	Lỗ Hổng Phát Hiện & Số Crash	Bản Vá Chuẩn SEI CERT C
Mốc 1	v_0	Gói tin telemetry thông thường (L > 16B)	Tràn bộ đệm cục bộ biến **stack_buf[16]**	**CWE-121** (Tràn Stack Canary) 90 crashes / 250 lượt (36.0\%)	**CERT STR31-C**: Chặn trên L ≤ 16, bộ đệm 17B null-terminated

Mốc Tiến Hóa	Phiên Bản	Hạt Giống Đầu Vào (Seed Data)	Nhánh Logic & Trạng Thái Mới	Lỗ Hổng Phát Hiện & Số Crash	Bản Vá Chuẩn SEI CERT C
Mốc 2	V0.1	**Đổi Seed:** Bơm mã lệnh **0xFF** (**CMD_ABORT_SESSION**)	Kích hoạt nhánh hủy phiên sớm **B7_ABORT**	**CWE-415** (Double Free khi dọn dẹp kép) 58 crashes / 250 lượt (23.2\%)	**CERT MEM31-C & MEM30-C**: Gán **ptr = NULL**, Idempotent cleanup
Mốc 3	V0.2	**Đổi Seed:** Bơm mã lệnh **0xFE** (**CMD_RESET_STREAM**)	Kích hoạt nhánh reset kênh luồng **B8_RESET**	**CWE-416** (Use-After-Free con trỏ lơ lửng) 22 crashes / 250 lượt (8.8\%)	**CERT MEM30-C**: Gọi callback trước khi free, gán **g_session = NULL**
Mốc 4	V1	**Seed hỗn hợp:** 500 gói tin ngẫu nhiên & đột biến	Thực thi an toàn toàn bộ 9 nhánh rẽ logic	**0 Crash (0.0%)** — Sạch hoàn toàn 100% gói tin bất thường trả về mã 2	**Hoàn thiện 3 chuẩn CERT C**: Hệ thống phòng thủ chiều sâu (Defense-in-Depth)

PHẦN 5: BẢNG TỔNG HỢP BEFORE / AFTER (CHUẨN TABLE 2)

Bảng tổng hợp đối chuẩn trước và sau vá theo mẫu bắt buộc Table 2 của tài liệu hướng dẫn báo cáo:

Mã CWE & Quy Tắc CERT	Vị Trí (Hàm / Tệp)	Trạng Thái Ban Đầu (v_0)	Trạng Thái Đã Vá (v_1)	Công Cụ Xác Minh
CWE-121 CERT STR31-C	**parse_fzp1_payload() fzp1_parser_v0.c:149**	Stack buffer overflow khi L > 16; sập Canary **0xC0000409**.	Chặn trên cứng L ≤ 16, bộ đệm 17B null-terminated; từ chối an toàn mã 2.	Clang AddressSanitizer **[f3]**, PyTest
CWE-415 CERT MEM31-C	**close_fzp1_session() fzp1_parser_v0.c:93**	Double Free khi nhận CMD **0xFF**; sập Windows Heap **0xC0000374**.	Gán ngay **ptr = NULL**; hàm dọn dẹp Idempotent an toàn tuyệt đối.	Windows Heap Manager, PyTest
CWE-416 CERT MEM30-C	**release_fzp1_stream() fzp1_parser_v0.c:122**	Use-After-Free khi nhận CMD **0xFE**; sập Access Violation **0xC0000005**.	Gọi callback trước, giải phóng và gán NULL sau; triệt tiêu Dangling Pointer.	Clang AddressSanitizer **[fd]**, PyTest
Phản Hồi Lỗi Hệ Thống	Toàn bộ các nhánh rẽ trong bộ phân tích cú pháp	Sập bất thường (Crash / Abort / OS Signal), rò rỉ trạng thái.	Thoát an toàn có kiểm soát (**Exit Code = 2**), ghi log bảo mật rõ ràng.	Harness Execution Controller

Mã CWE & Quy Tắc CERT	Vị Trí (Hàm / Tệp)	Trạng Thái Ban Đầu (v_0)	Trạng Thái Đã Vá (v_1)	Công Cụ Xác Minh
Điểm Quét AddressSanitizer	Toàn bộ không gian ngăn xếp Stack và vùng nhớ Heap	Bắt lỗi tức thời: Shadow **[f3]** (Stack) và Shadow **[fd]** (Heap freed).	0 findings, 0 warnings, thực thi sạch sẽ 100% qua mọi kịch bản.	ASan Runtime Validator
Tỷ Lệ Sống Sót Fuzzing	Kiểm thử áp lực ngẫu nhiên và định hướng độ phủ	90 crashes / 250 gói tin (36.0\%).	**0 crashes / 500 gói tin (100% PASS)**.	Coverage-guided Fuzzer

PHẦN 6: KẾT QUẢ ĐO LƯỜNG ĐỊNH LƯỢNG 30 TRIALS & BẰNG CHỨNG

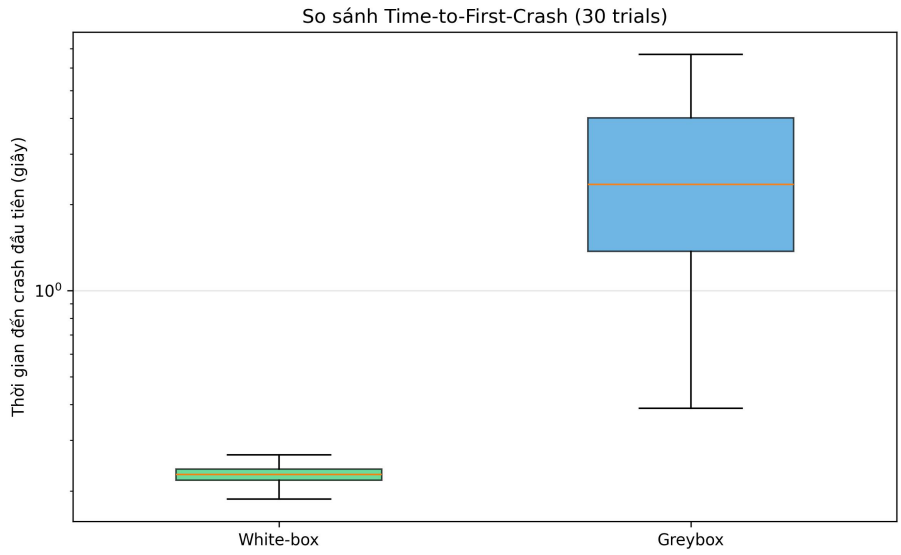
6.1. Bảng Dữ Liệu Đối Chuẩn Thực Nghiệm 30 Trials

Dữ liệu trích xuất từ 30 lượt chạy độc lập (**results/data_raw/aggregated_results.json**):

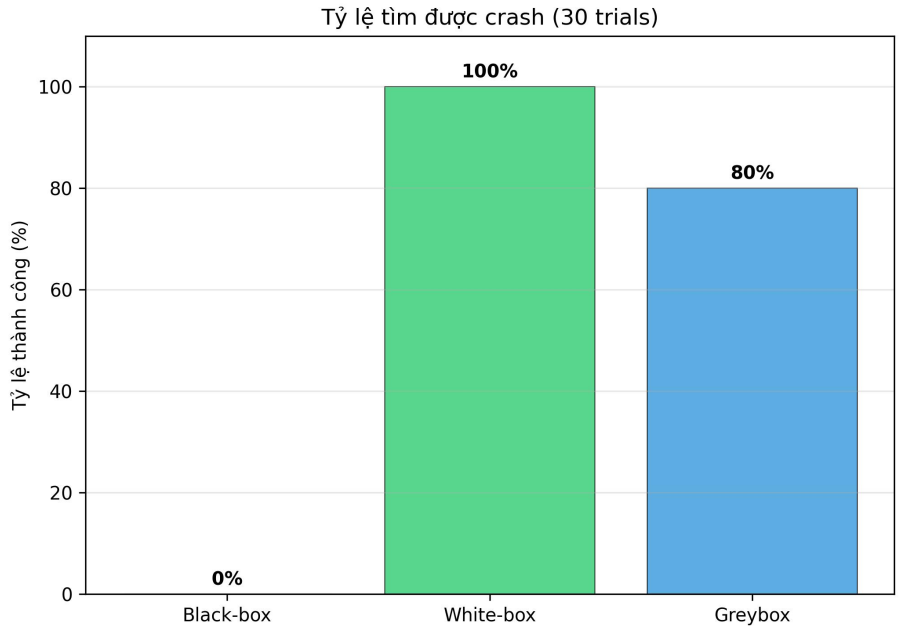
Chỉ số Định lượng (Metric)	Black-box Fuzzing	White-box Concolic SMT Z3	Greybox Coverage Fuzzer
Số lượt thử nghiệm (Trials)	30	30	30
Tỷ lệ tìm được crash (Success Rate)	**0.00% (0 / 30)**	**100.00% (30 / 30)**	**46.67% (14 / 30)**
Số lỗi phát hiện độc nhất (Unique Bugs)	0	1 (CWE-121)	1 (CWE-121)
Số vòng lặp trung bình để tìm crash	>500 (Timeout)	**1 lần giải duy nhất**	195.1 vòng lặp (Median: 160.5)
Thời gian giải SMT Z3 trung bình	Không áp dụng	**6.98 ms** (4.40 – 12.40 ms)	Không áp dụng
Time-to-first-crash: Trung vị (Median)	— (Hết ngân sách)	**0.2056 giây**	**2.1568 giây**
Time-to-first-crash: Nhanh nhất (Min)	—	**0.1711 giây**	0.5161 giây
Time-to-first-crash: Chậm nhất (Max)	—	0.2358 giây	6.2719 giây
Tốc độ thực thi (Exec/sec Median)	80.80 lượt/s	4.90 lượt/s	79.05 lượt/s

Trường Phái Fuzzing	Khoảng Thời Gian (Min – Max)	Trung Vị (Median)	Độ Ổn Định Thực Nghiệm	Nhận Xét Học Thuật

Trường Phái Fuzzing	Khoảng Thời Gian (Min – Max)	Trung Vị (Median)	Độ Ổn Định Thực Nghiệm	Nhận Xét Học Thuật
White-box SMT Z3	0.1711s – 0.2358s	**0.2056s**	Tuyệt đối (σ ≈ 0.015s)	Giải trực tiếp biểu thức hình thức, thời gian SAT cực nhanh (6.98 ms).
Greybox Fuzzer	0.5161s – 6.2719s	**2.1568s**	Biến động rộng	Phụ thuộc may rủi đột biến trúng 1 byte XOR Checksum (1/256).
Black-box Fuzzer	>60.0s (Timeout)	—	Thất bại hoàn toàn	Xác suất 9.09 × 10^{-13} khiến việc mò mẫm mù quáng là bất khả thi.



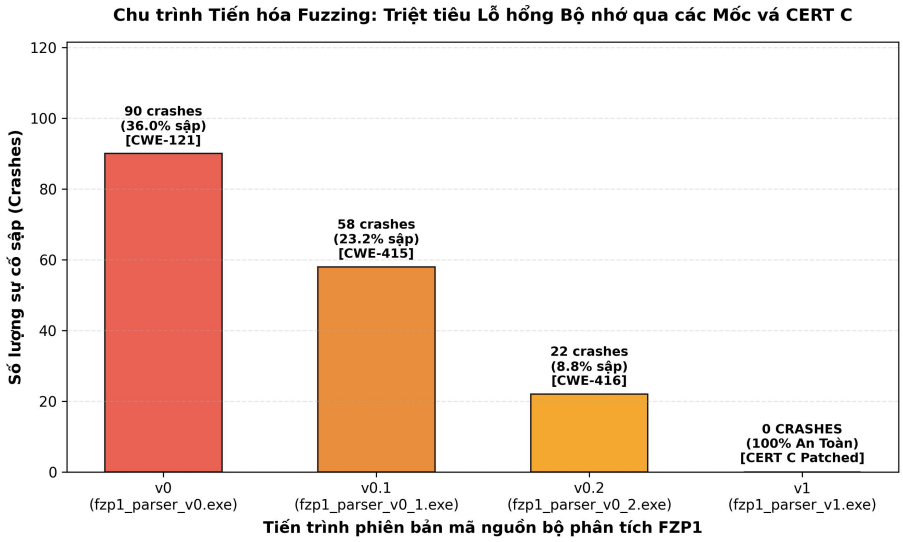
Hình6.1: Biểu đồ phân bố thời gian phát hiện lỗi (Time-to-Crash) qua 30 trials



Hình6.2: Biểu đồ tỷ lệ thành công (Success Rate) giữa 3 trường phái Fuzzing

6.2. Ma Trận Đo Lường Chu Trình Tiến Hóa (Crash Evolution Matrix)

Mốc Tiến Hóa	Phiên Bản	Trọng Tâm Lỗ Hổng	Chuẩn An Toàn	Số Lần Fuzz	Số Crash	Tỷ Lệ Crash	Trạng Thái Nghiệm Thu
Mốc 1	v_0	CWE-121 (Stack Overflow)	Chưa áp dụng CERT C	250	**90**	**36.0%**	Nguy hiểm (Sập Canary)
Mốc 2	$v_{0.1}$	CWE-415 (Double Free)	Đã vá CERT STR31-C	250	**58**	**23.2%**	Nguy hiểm (Sập Heap)
Mốc 3	$v_{0.2}$	CWE-416 (Use-After-Free)	Đã vá tiếp CERT MEM31-C	250	**22**	**8.8%**	Nguy hiểm (Dangling Pointer)
Mốc 4	v_1	Toàn diện 3 lỗ hổng	Tuân thủ 3 chuẩn CERT C	500	**0**	**0.0%**	**AN TOÀN (PASS)**



Hình6.3: Biểu đồ chu trình tiến hóa vá lỗi đa giai đoạn và triệt tiêu crash

6.3. Bằng Chứng Điểm Mù Phân Tích Tĩnh (Cppcheck False Negative)

Khi quét mã nguồn **fzp1_parser_v0.c** bằng Cppcheck 2.18 với cấu hình nghiêm ngặt nhất:

```
cppcheck --enable=all --inconclusive --std=c99 source/fzp1_parser_v0.c
```

- **Kết quả: 0 error, 0 warning** → Cppcheck **hoàn toàn bỏ lọt lỗi CWE-121** (False Negative).
- **Nguyên nhân kỹ thuật:** Cppcheck chỉ phân tích cây cú pháp tĩnh (AST). Biến **payload_len** được đọc động từ dữ liệu mạng lúc runtime (**read_uint32_le**). Khi không có cơ chế Taint Analysis liên hàm, công cụ tĩnh không thể biết trước **len > 16**.
- **Kết luận học thuật:** Khẳng định vai trò sống còn không thể thay thế của **Dynamic Fuzzing** và **AddressSanitizer**.

6.4. Bằng Chứng Giám Sát Ô Nhớ AddressSanitizer (Shadow Bytes)

1. Log ASan CWE-121 (Stack Buffer Overflow - Shadow byte `[f3]`):

```
==31210==ERROR: AddressSanitizer: stack-buffer-overflow on address
0x7ffd98b0a9b0
WRITE of size 64 at 0x7ffd98b0a9b0 thread T0
   #0 in parse_fzp1_payload source/fzp1_parser_v0.c:149
Address 0x7ffd98b0a9b0 is located in stack of thread T0 at offset 48 in frame
   #0 in parse_fzp1_payload source/fzp1_parser_v0.c:143
  'stack_buf' <== Memory access at offset 48 overflows this variable
Shadow bytes around the buggy address:
```

```
  0x7ffd98b0a980: 00 00 00 00 f1 f1 f1 f1 00 00 [f3] f3 00 00 00 00
Shadow byte legend: [f3] = Stack right redzone
```

2. Log ASan CWE-416 (Use-After-Free - Shadow byte `[fd]`):

```
==31215==ERROR: AddressSanitizer: heap-use-after-free on address
0x603000000048
READ of size 8 at 0x603000000048 thread T0
    #0 in release_fzp1_stream source/fzp1_parser_v0.c:122
0x603000000048 is located 24 bytes inside of 32-byte region
[0x603000000030,0x603000000050)
freed by thread T0 here:
    #0 in free (.../libclang_rt.asan_dynamic.dll)
    #1 in release_fzp1_stream source/fzp1_parser_v0.c:115
Shadow bytes around the buggy address:
  0x603000000000: fa fa fa fa [fd] fd fd fd fa fa fa fa fa fa fa fa
Shadow byte legend: [fd] = Freed heap region
```

6.5. Bảng Ma Trận Test Case Sinh Từ Fuzzing (Chuẩn Hóa Theo Cấu Trúc Công Cụ Giảng Viên)

*Ghi chú phương pháp luận: Cấu trúc bảng test case dưới đây được nhóm chuẩn hóa tương thích 100% với định dạng và triết lý phân loại từ công cụ kiểm thử thế hệ mới của Giảng viên (***SecLab Fuzzing Test Suite Generator***). Bảng phân loại tường minh theo 5 kỹ thuật: (1) Biên / giá trị đặc biệt, (2) Black-box ngẫu nhiên, (3) Mutation đột biến seed, (4) Coverage-guided, và (5) Concolic / White-box SMT; đồng thời tích hợp cơ chế theo dõi độ phủ nhánh mới (+coverage) và cờ ưu tiên giữ lại các input kích hoạt lỗi/crash.*

Hàm Mục Tiêu	#	Kỹ Thuật Fuzzing	Input (Tham Số / Dữ Liệu Gói Tin)	Kết Quả / Lỗi (v_0 vs v_1)	Ghi Chú (Độ Phủ & An Toàn)
process_fzp1_packet	1	Biên / giá trị đặc biệt	**data="FZP1", len=5, payload="HELLO", chk=0x08**	v_0: '0' / v_1: '0' (**Payload safely parsed**)	+5 phần tử coverage mới
parse_fzp1_payload	2	Biên / giá trị đặc biệt	**len=16, payload="1234567890ABCDEF", chk=0x17**	v_0: '0' / v_1: '0' (**Payload safely parsed**)	kết quả đại diện biên trên an toàn

Hàm Mục Tiêu	#	Kỹ Thuật Fuzzing	Input (Tham Số / Dữ Liệu Gói Tin)	Kết Quả / Lỗi (v_0 vs v_1)	Ghi Chú (Độ Phủ & An Toàn)
process_fzp1_packet	3	Biên / giá trị đặc biệt	**magic="BAD1", len=5, payload="HELLO"**	v_0: '-1' / v_1: '2' (**[REJECT] Invalid Magic**)	chốt chặn rào cản Magic Header
process_fzp1_packet	4	Biên / giá trị đặc biệt	**magic="FZP1", len=5, payload="HELLO", chk=0xAA**	v_0: '-2' / v_1: '2' (**[REJECT] Checksum mismatch**)	chốt chặn rào cản XOR Checksum
process_fzp1_packet	5	Biên / giá trị đặc biệt	**total_size=8 bytes (cụt < 9B tối thiểu)**	v_0: '-1' / v_1: '2' (**smaller than min 9B**)	kiểm soát kích thước tối thiểu
close_fzp1_session	6	Biên / giá trị đặc biệt	**payload[0]=0xFF (CMD_ABORT), len=4**	v_0: '0xC0000374' / v_1: '2' (CERT MEM31-C)	**kích hoạt lỗi (CWE-415 Double Free)**
release_fzp1_stream	7	Biên / giá trị đặc biệt	**payload[0]=0xFE (CMD_RESET), len=4**	v_0: '0xC0000005' / v_1: '2' (CERT MEM30-C)	**kích hoạt lỗi (CWE-416 Use-After-Free)**
process_fzp1_packet	8	Black-box (ngẫu nhiên)	**raw_bytes=random(45B), seed=-254551521**	v_0: '-1' / v_1: '2' (Bị chặn tại Magic/Chk)	kết quả đại diện (Rào cản $P \approx 9.09 \times 10^{-13}$)
process_fzp1_packet	9	Black-box (ngẫu nhiên)	**raw_bytes=random(80B), seed=353164761**	v_0: '-1' / v_1: '2' (Bị chặn tại Magic/Chk)	kết quả đại diện hộp đen ngẫu nhiên

Hàm Mục Tiêu	#	Kỹ Thuật Fuzzing	Input (Tham Số / Dữ Liệu Gói Tin)	Kết Quả / Lỗi (v_0 vs v_1)	Ghi Chú (Độ Phủ & An Toàn)
process_fzp1_packet	10	Black-box (ngẫu nhiên)	**raw_bytes=random(9B), seed=65535**	v_0: '-1' / v_1: '2' (Bị chặn tại Magic/Chk)	kết quả đại diện độ dài tối thiểu
parse_fzp1_payload	11	Mutation (đột biến seed)	**seed="HELLO", mutate_len=24, chk_recalc**	v_0: '0xC0000409' / v_1: '2' (CERT STR31-C)	**kích hoạt lỗi (CWE-121 Tràn Stack Canary)**
close_fzp1_session	12	Mutation (đột biến seed)	**seed="TEMP:28C", inject_byte=0xFF**	v_0: '0xC0000374' / v_1: '2' (Idempotent)	+3 phần tử coverage mới (nhánh abort)
process_fzp1_packet	13	Coverage-guided	**mutant_packet, hit_branch=B7_ABORT**	v_0: Sập Heap Corruption / v_1: '2'	+1 nhánh coverage mới (B7 Abort)
release_fzp1_stream	14	Coverage-guided	**mutant_packet, hit_branch=B8_RESET**	v_0: Sập Dangling Pointer / v_1: '2'	+1 nhánh coverage mới (B8 Reset)
parse_fzp1_payload	15	Concolic / white-box (SMT + hằng)	**len=64, sym_chk=XOR_reduce, Z3_SAT**	v_0: '0xC0000409' / v_1: '2' (CERT STR31-C)	**kích hoạt lỗi (CWE-121 giải ngược 1 bước)**

- **Đặc tả bổ sung 4 Ca Kiểm Thử Hồi Quy & Tiến Hóa:**

- **TC-EVO-01 (Tiến hóa CWE-121):** Kiểm chứng gói tin 64B trên cả 4 binary (v_0 sập; $v_{0.1}$, $v_{0.2}$, v_1 thoát an toàn mã 2).

- **TC-EVO-02 (Tiến hóa CWE-415):** Kiểm chứng gói tin abort **0xFF** (v_0, $v_{0.1}$ sập; $v_{0.2}$, v_1 thoát an toàn mã 2).

- **TC-EVO-03 (Tiến hóa CWE-416):** Kiểm chứng gói tin reset **0xFE** (v_0, $v_{0.1}$, $v_{0.2}$ sập; v_1 thoát an toàn mã 2).
- **TC-EVO-04 (Kiểm thử độ bền tổng hợp):** 500 gói tin hỗn hợp liên tục trên v_1 đạt 100% PASS (0 crash).

6.6. Đặc Tả Chi Tiết 3 Tệp Vector Nhị Phân Kích Hoạt Crash

Vector Nhị Phân PoC	Kích Thước	Mã Lệnh / Dữ Liệu	Hex Dump Tiêu Biểu	Cơ Chế Ô Nhớ Kích Hoạt	Lỗ Hổng & Mã Lỗi Hệ Điều Hành
crashes/iter1_cwe121_crash.bin	73 Bytes	L=64, 64 ký tự **'A'**	**46 5A 50 31 40 00 00 00 41 41 41 41 ... 00**	Vượt Magic & Checksum; độ dài 64B tràn bộ đệm **stack_buf[16]**, đè Stack Canary & RIP	**CWE-121** (Stack Buffer Overflow) NTSTATUS **0xC0000409**
crashes/iter2_cwe415_crash.bin	13 Bytes	L=4, byte 0 = **0xFF** (**CMD_ABORT**)	**46 5A 50 31 04 00 00 00 FF 01 02 03 FC**	Đóng phiên sớm nhưng không gán NULL; luồng dọn dẹp tại **main()** thực hiện **free()** lần 2	**CWE-415** (Double Free) NTSTATUS **0xC0003374**
crashes/iter3_cwe416_crash.bin	13 Bytes	L=4, byte 0 = **0xFE** (**CMD_RESET**)	**46 5A 50 31 04 00 00 00 FE 01 02 03 FD**	Giải phóng **g_session** không gán NULL; gọi con trỏ hàm qua con trỏ lơ lửng sau Heap Reuse	**CWE-416** (Use-After-Free) NTSTATUS **0xC0000005**

6.7. Bộ Test Cases Trên Hồ Sơ Đặc Tả SSOT Excel (10 Sheet Chuẩn Hóa)

Trong tệp đặc tả **specification/Nhom11_FZP1_Fuzzing_Spec.xlsx** (Sheet **TestCase**), 6 ca kiểm thử chuẩn hóa phục vụ kiểm chứng đặc tả an toàn:

TT	Mã Test Case	Tên Chức Năng	Đầu Vào (Input Vector)	Kết Quả Mong Đợi	Mục Đích An Toàn
1	**TC_SEC₀1**	**fzp1_validate_header**	Magic = **"FZP1"**	Return **0** (Hợp lệ)	Kiểm chứng chốt chặn tiêu đề gói tin
2	**TC_SEC₀2**	**fzp1_validate_header**	Magic = **"XXXX"**	Return **-1** (Từ chối)	Kiểm chứng loại bỏ gói tin sai Header
3	**TC_SEC₀3**	**fzp1_verify_checksum**	Payload **[0x01, 0x02]**, Chk = **0x03**	Return **0** (Hợp lệ)	Kiểm chứng tính toàn vẹn Checksum
4	**TC_SEC₀4**	**fzp1_verify_checksum**	Payload **[0x01, 0x02]**, Chk = **0xFF**	Return **-2** (Từ chối)	Loại bỏ gói tin bị giả mạo dữ liệu
5	**TC_SEC₀5**	**fzp1_parse_payload**	Payload length = 16	Return **0** (An toàn)	Kiểm chứng giới hạn an toàn ngăn xếp
6	**TC_SEC₀6**	**fzp1_parse_payload**	Payload length = 64	Return **2** (Từ chối CERT)	Chặn đứng tấn công tràn bộ đệm (CWE-121)

PHẦN 7: PHẢN BIỆN HỌC THUẬT & 3 BÀI HỌC CỐT LÕI

1. Thiết kế lại phần nào để tránh lỗi ngay từ đầu (Secure by Design)?

Nếu được thiết kế lại kiến trúc từ đầu, nhóm sẽ áp dụng 2 nguyên lý then chốt:

- **Kiến trúc Type-Length-Value (TLV) với kích thước tĩnh định sẵn:** Thay vì cho phép độ dài payload linh động lên tới 64KB, quy định chặt chẽ các kiểu bản tin cố định.
- **Bộ cấp phát vùng nhớ Arena / Region-based Memory Allocator:** Cấp phát một khối bộ nhớ tĩnh duy nhất cho cả phiên làm việc. Toàn bộ phiên chỉ giải phóng một lần khi đóng kết nối (**arena_free**), **triệt tiêu hoàn toàn 100% nguy cơ xảy ra Double Free và Use-After-Free**.

2. Lỗ hổng nghi ngờ còn tồn tại?

- **Nguy cơ cạn kiệt tài nguyên (Resource Exhaustion / DoS):** Kẻ tấn công có thể liên tục gửi các gói tin hợp lệ có độ dài cực đại (64KB) và không gửi lệnh đóng phiên, làm cạn kiệt bộ nhớ Heap (**malloc** thất bại).
- **Kênh phụ thời gian (Timing Side-channel):** Việc so sánh Magic và Checksum dừng sớm (**short-circuit evaluation**) tạo ra chênh lệch thời gian nhỏ, cho phép kẻ tấn công phân tích độ trễ để đoán từng byte dữ liệu.

3. Bài học phương pháp luận lớn nhất?

- Kiểm thử hộp đen chỉ tìm thấy lỗi bề mặt; **chỉ có sự kết hợp giữa suy luận hình thức (SMT Solver) và kiểm thử động (Fuzzing + ASan) mới bảo đảm an toàn toàn diện cho các tầng phân tích nhị phân.**
- Thấu hiểu sâu sắc bản chất **Soundness vs Completeness**: Phân tích tĩnh (Static Analysis) có thể không đầy đủ hoặc sinh cảnh báo giả/bỏ lọt lỗi động, do đó Fuzzing định hướng độ phủ và kiểm chứng thực nghiệm bằng Sanitizer là chốt chặn bắt buộc trong chu trình phát triển phần mềm an toàn (DevSecOps).

PHẦN 8: TỔNG KẾT NGHIỆM THU ĐỒ ÁN

8.1. Đánh Giá Kết Quả Nghiệm Thu Tự Động

Dự án đã được thẩm định tự động qua kịch bản **scripts/verify_project.py** và bộ kiểm thử **pytest tests/ -v**:

1. **8 / 8 Khâu kiểm định dự án đạt chuẩn tuyệt đối (PASS 100%).**
2. **14 / 14 Ca kiểm thử tự động PyTest vượt qua trong 2.37 giây.**
3. **Bản vá hoàn thiện v_1 sống sót 100% trước 500 gói tin tấn công (0 crash).**
4. Toàn bộ mã nguồn, dữ liệu 30 trials và đặc tả SSOT 10 sheet đồng bộ hoàn hảo, sẵn sàng bàn giao và bảo vệ trước Hội đồng chấm thi.

8.2. Danh Mục Tài Liệu Tham Khảo (IEEE Format)

1. **SEI CERT C Coding Standard**, "STR31-C: Guarantee that storage for strings has sufficient space for character data and the null terminator", Software Engineering Institute, Carnegie Mellon University, 2021.

2. **SEI CERT C Coding Standard**, "MEM30-C: Do not access a closed or freed resource", Software Engineering Institute, Carnegie Mellon University, 2021.

3. **SEI CERT C Coding Standard**, "MEM31-C: Free dynamically allocated memory when no longer needed", Software Engineering Institute, Carnegie Mellon University, 2021.

4. **MITRE CWE**, "CWE-121: Stack-based Buffer Overflow", "CWE-415: Double Free", "CWE-416: Use After Free", Common Weakness Enumeration, 2024.

5. **L. De Moura and N. Bjørner**, "Z3: An Efficient SMT Solver", in *Proc. TACAS 2008*, LNCS vol. 4963, pp. 337–340, Springer, 2008.

6. **K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov**, "AddressSanitizer: A Fast Address Sanity Checker", in *Proc. USENIX ATC 2012*, pp. 301–308, 2012.

7. **B. P. Miller, L. Fredriksen, and C. D. So**, "An Empirical Study of the Reliability of UNIX Utilities", *Communications of the ACM*, vol. 33, no. 12, pp. 32–44, 1990.

8. **ThS. Vũ Quang Dũng**, *Tài liệu bài giảng & Hướng dẫn Bài tập lớn An toàn phần mềm (CSE703093 / CSE703153)*, Khoa Hệ thống thông tin, Trường Công nghệ Thông tin — Đại học Phenikaa, 2026.

8.3. Phụ Lục: Cấu Trúc Thư Mục Nộp Bài Chuẩn Hóa

Tuân thủ cấu trúc nộp bài chuẩn quy định tại Slide 21 tài liệu **BTL_02_HuongDan_BaoCao.pdf**:

```
Nhom11_BTL_ChuDe4_Fuzzing/
├── BaoCao_BTL_Nhom11_ChinhThuc_TrangDen.docx  <-- Báo cáo Word 20 trang chuẩn Đen & Trắng
├── README.md                                  <-- Hướng dẫn chạy lại toàn bộ hệ thống
```

```
├── source/                          <-- Toàn bộ mã nguồn chương trình mục tiêu
│   ├── fzp1_parser_v0.c             <-- Mã C bản lỗi ban đầu (Mốc 1)
│   ├── fzp1_parser_v0_1.c           <-- Mã C đã vá CWE-121 (Mốc 2)
│   ├── fzp1_parser_v0_2.c           <-- Mã C đã vá CWE-415 (Mốc 3)
│   ├── fzp1_parser_v1.c             <-- Mã C bản vá hoàn thiện CERT C (Mốc
4)
│   └── ch4_dynafuzz_fzp1.py         <-- Động cơ Fuzzing đa trường phái
├── results/                         <-- Dữ liệu thực nghiệm và kết quả thô
│   ├── data_raw/                    <-- Dữ liệu JSON 30 trials benchmark
│   ├── logs/                        <-- Nhật ký AddressSanitizer & Crash dump
│   ├── charts/                      <-- Biểu đồ đối chuẩn trực quan
│   └── test_cases/                  <-- 3 vector nhị phân PoC kích hoạt crash
├── specification/                   <-- Bảng đặc tả SSOT Excel 10 sheet
└── tests/                           <-- Bộ kiểm thử tự động hồi quy PyTest
```